

Project KAIROS — Detailed Build Plan

A Probabilistic Consensus-State Server with Observer-Driven Collapse

Version 1.0

0. Analysis — What This Actually Is

In one paragraph: KAIROS is a distributed state backend where entity properties are stored as probability distributions rather than scalar values. A synchronous serving loop answers queries against these distributions. An asynchronous consensus engine monitors observations; when observer count and confidence cross thresholds, the system collapses the distribution to a deterministic value via weighted consensus. The deliverable is a measurable claim: on synthetic and real workloads, probabilistic storage reduces stale-read rate and prediction latency versus deterministic cache-and-invalidate, at matched consistency budgets.

Where the value concentrates: 70% of credibility lives in Stages 0-2 — proving that the probabilistic engine (a) converges to deterministic behaviour under certainty, (b) never loses data, and (c) beats a Redis baseline on stale-read rate. The “quantum” metaphor is UX; the engineering is Bayesian consensus plus event sourcing.

The four ways it dies (each has a gate)

#	Failure mode	Gate
1	It's just a buggy cache. Probabilistic reads return garbage and you can't prove otherwise.	Deterministic convergence test against Redis (Stage 2)
2	Consensus never converges. Observers disagree forever, state stays fuzzy, users get nonsense.	Collapse latency histogram + timeout-bound fallback (Stage 3)
3	It's slower than just hitting Postgres. Probabilistic math and consensus overhead dominate any benefit.	Latency-throughput comparison vs Redis + Postgres at matched consistency (Stage 4)
4	Event sourcing loses events. The async collapse engine drops observations, creating silent divergence.	Exactly-once semantics + determinism replay (Stage 5)

1. Ground Rules — Non-Negotiable

- Deterministic baseline first.** No probabilistic code until a Redis-backed deterministic API exists and is load-tested. The baseline is the control; KAIROS is the treatment.
- Collapse must converge.** Every entity has a `max_observation_age` and `max_uncertainty`. Breach either → deterministic fallback (last-writer-wins with vector clock). The system is never allowed to stay “fuzzy” forever.
- No model on the serving path.** Loop 1 (FastAPI) reads from Redis/Cache. Loop 2 (Java consensus engine) updates probabilities asynchronously. If the learner dies, reads degrade to last-known-deterministic, not chaos.
- Time-based event log only.** Observations are append-only. No in-place updates. No random-access mutation.
- Matched comparison windows.** Any claim (“23% fewer stale reads”) compares identical request sequences against identical backends, same warmup, same cache size.
- Report the harm.** Fuzzy reads, unresolved conflicts, fallback rate and consensus latency are first-class metrics, printed beside the headline.
- Boring stack, except where justified.** Python (API), Java (consensus engine), Rust (probabilistic data structures). Redis for state. Kafka for event log. Docker Compose for orchestration. Any new dependency needs a one-line justification in `decisions.md`.
- Stage gates are hard.** Each stage ends in an acceptance checklist. All green before the next stage.
- Everything reproducible.** Every result JSON records git SHA, seed, protocol and parameters.

10. **Name it what it is.** The system is a probabilistic consensus-state layer. The words “quantum” and “superposition” appear in the README and demo only, never in code or claims. The physics metaphor explains the architecture; it does not excuse hand-waving.

2. Repository, Environment, and Conventions

2.1 Repository Layout

```
kairos/
├── README.md           # claims + architecture diagram + protocol
├── PLAN.md             # this document
├── CLAUDE.md           # AI agent instructions
├── decisions.md        # running log: decision / options / why
├── Makefile
├── requirements.txt
├── pyproject.toml
├── docker/
│   ├── Dockerfile.python    # API layer
│   ├── Dockerfile.java      # Consensus engine
│   ├── Dockerfile.rust      # Probabilistic FFI lib
│   └── docker-compose.yml
├── data/
│   ├── raw/                # synthetic traces, real logs – gitignored
│   └── processed/          # parquet traces – gitignored
├── src/kairos/
│   ├── __init__.py
│   ├── config.py           # single dataclass, every knob
│   ├── api.py              # FastAPI serving loop
│   ├── models.py           # Pydantic: ProbabilityCloud, CollapsedState
│   ├── policies/
│   │   ├── base.py         # StateBackend contract
│   │   ├── deterministic.py # Redis baseline (scalar values)
│   │   └── probabilistic.py # KAIROS backend (distributions)
│   ├── harness/
│   │   ├── trace_replay.py  # deterministic load generator
│   │   ├── metrics.py       # stale-read rate, latency, conflict rate
│   │   └── validator.py     # convergence checker vs Redis
│   └── plots/
│       └── report.py
├── consensus/            # Java – the collapse engine
│   ├── src/main/java/kairos/
│   │   ├── engine/
│   │   │   ├── ConsensusEngine.java
│   │   │   ├── BayesianCollapse.java
│   │   │   └── TemporalDecay.java
│   │   ├── store/
│   │   │   ├── ObservationLog.java
│   │   │   └── ProbabilityTable.java
│   │   └── transport/
│   │       ├── KafkaConsumer.java
│   │       └── RedisPublisher.java
│   ├── pom.xml
│   └── Dockerfile
├── prob/                  # Rust – probabilistic data structures
│   ├── src/
│   │   ├── lib.rs
│   │   ├── amplitude.rs    # amplitude vectors
│   │   ├── sketch.rs       # Count-Min, HyperLogLog
│   │   └── ffi.rs          # JNI / C ABI exports
│   ├── Cargo.toml
│   └── Dockerfile
├── service/              # Live demo frontend
│   ├── static/
│   │   ├── index.html
│   │   ├── cloud.html      # real-time probability viz
│   │   └── consensus.html  # observer/collapse dashboard
│   └── demo_driver.py
├── tests/                # pytest + JUnit + Rust tests
└── results/              # JSON + PNG artifacts, committed
```

2.2 Environment and Dependencies

Python 3.11+

```
fastapi
uvicorn[standard]
redis
httpx
numpy
pandas
pyarrow
matplotlib
pytest
kafka-python
pydantic
```

Java 17+, Maven — kafka-clients, redis-lettuce, jackson, junit5, slf4j.

Rust 1.75+

```
[dependencies]
probabilistic-collections = "0.7"
serde = { version = "1.0", features = ["derive"] }
libc = "0.2"
```

2.3 Makefile — The Public API

```
.PHONY: setup build test data baselines probabilistic sweep plots demo deploy

setup:
    pip install -r requirements.txt && pip install -e .
    cd consensus && mvn clean package -DskipTests
    cd prob && cargo build --release

build:
    docker compose -f docker/docker-compose.yml build

test:
    pytest -q
    cd consensus && mvn test
    cd prob && cargo test

data:
    python -m kairos.harness.trace_replay --generate --out data/processed/

baselines:
    python -m kairos.harness.trace_replay \
        --trace data/processed/synthetic.parquet \
        --backend deterministic --protocol P1 --out results/

probabilistic:
    python -m kairos.harness.trace_replay \
        --trace data/processed/synthetic.parquet \
        --backend probabilistic --protocol P1 \
        --tau 0.6 --observers 3 --out results/

sweep:
    python -m kairos.harness.trace_replay \
        --trace data/processed/synthetic.parquet \
        --sweep-tau --sweep-observers --out results/

plots:
    python -m kairos.plots.report --results results/ --out results/

demo:
    docker compose -f docker/docker-compose.yml up -d

deploy:
    # Stage 8 target
```

3. Stage 0 — Data and Trace Generation (1 day)

Goal: a reproducible, seeded synthetic trace generator that creates realistic state-mutation patterns. This is the “production log” for offline evaluation.

3.1 Trace Schema

Each row represents an observation event:

```

class ObservationEvent:
    ts_ms: int          # timestamp
    seq: int            # global sequence
    entity_id: str       # e.g., "user:42", "driver:99"
    attribute: str       # e.g., "location", "status", "mood"
    value: str          # observed scalar value
    observer_id: str     # e.g., "api-gateway-1", "mobile-app"
    confidence: float    # 0.0–1.0, observation reliability
    session_id: str      # for entanglement context

```

3.2 Synthetic Generator

`src/kairos/harness/trace_replay.py --generate`

Three user personas, each with transition matrices:

Persona	Behaviour
Commuter	<code>location</code> cycles: home → transit → work → transit → home. High predictability.
Shopper	<code>intent</code> jumps: browse → compare → cart → checkout/abandon. Medium predictability.
Chaotic	Random attribute flips. Low predictability — tests fallback behaviour.

Rules

- Seed everything (`numpy.random.default_rng(seed=42)`).
- Generate 1M events across 1000 entities, 30-day span.
- 80% Commuter, 15% Shopper, 5% Chaotic.
- Burst mode: every 1000 events, inject a 10-event burst to the same entity (simulates thundering herd).
- Output: `synthetic.parquet` with the schema above.

3.3 Entanglement Graph

Generate a static `entanglements.json`:

```

{
  "product-A": ["product-B", "product-C"],
  "user:42": ["session:42-cart", "session:42-wishlist"]
}

```

Used in Stage 5. 10% of entities have 1–3 entangled partners.

Acceptance — Stage 0

- ☐ `make data` runs idempotently; same seed → identical parquet (byte check).
- ☐ `make stats` prints total events, unique entities, unique observers, event rate/sec, burst distribution.
- ☐ `decisions.md` records persona mix rationale, seed value, entanglement density.

4. Stage 1 — Deterministic Baseline (1-2 days)

Goal: a correct, load-tested deterministic state API backed by Redis. This is the control group.

4.1 The StateBackend Contract

`src/kairos/policies/base.py`

```

class StateBackend(ABC):
    name: str = "base"

    @abstractmethod
    async def get(self, entity_id: str, attribute: str) -> CollapsedState:
        """Return deterministic value + metadata."""

    @abstractmethod
    async def put(self, entity_id: str, attribute: str, value: str,
                  observer_id: str, confidence: float) -> None:
        """Write observation. For deterministic: overwrite."""

    @abstractmethod
    async def observe(self, event: ObservationEvent) -> None:
        """Ingest observation into backend."""

    def stats(self) -> dict:
        """Return internal counters."""

```

4.2 Deterministic Backend

src/kairos/policies/deterministic.py

- Redis hash per entity: HSET kairos:det:{entity_id} {attribute} {value}
- TTL: 300s default.
- On get: read hash, return CollapsedState(value, certainty=1.0, collapsed_at=ts).
- On put: overwrite hash, publish invalidation to the kairos:invalidations stream.

4.3 FastAPI Serving Loop

src/kairos/api.py

```

@app.get("/entities/{entity_id}")
async def get_entity(entity_id: str, backend: StateBackend = Depends(...)):
    state = await backend.get(entity_id, "status")
    return state

@app.post("/observe")
async def observe(event: ObservationEvent, backend: StateBackend = Depends(...)):
    await backend.observe(event)
    return {"ok": True}

```

4.4 Tests

1. **Determinism** — same sequence of puts → same get result, always.
2. **TTL expiry** — after 300s of silence, get returns None (cache miss).
3. **Concurrency** — 100 concurrent puts to the same key, last writer wins, no corruption.
4. **Latency** — p99 get < 5ms against local Redis (baseline for later comparison).

Acceptance — Stage 1

- ☐ make baselines runs the full 1M-event trace through the deterministic backend, prints hit rate and latency.
- ☐ All four tests green.
- ☐ Full trace replay < 60s.

5. Stage 2 — Probabilistic Core and Convergence Validation (2 days)

Goal: the Rust probabilistic data structures are wired into Python. Prove that under certainty (confidence=1.0, single observer) KAIROS converges to identical behaviour to the deterministic backend.

5.1 Rust FFI Layer

prob/src/

```
// AmplitudeVector: stores weighted hypotheses for one attribute
pub struct AmplitudeVector {
    pub hypotheses: HashMap<String, f64>, // value -> weight
    pub total_observations: u32,
    pub last_updated_ms: u64,
}

impl AmplitudeVector {
    pub fn observe(&mut self, value: String, confidence: f64) {
        // Bayesian update: weight *= (1-confidence); then boost observed
        let boost = 1.0 + confidence * 2.0;
        *self.hypotheses.entry(value).or_insert(0.1) *= boost;
        self.normalize();
    }

    pub fn collapse(&self, threshold: f64) -> Option<String> {
        // Return mode if its probability >= threshold
        let (mode, prob) = self.hypotheses.iter()
            .max_by(|a, b| a.1.partial_cmp(b.1).unwrap());
        if *prob >= threshold { Some(mode.clone()) } else { None }
    }
}
```

Expose via C ABI:

```
#[no_mangle]
pub extern "C" fn amplitude_vector_new() -> *mut AmplitudeVector { ... }

#[no_mangle]
pub extern "C" fn amplitude_vector_observe(ptr: *mut AmplitudeVector,
                                           value: *const c_char,
                                           confidence: c_double) { ... }
```

5.2 Python Probabilistic Backend

src/kairos/policies/probabilistic.py

- Maintains `prob:{entity_id}:{attribute}` in Redis as a JSON blob (the amplitude vector).
- `get`: read blob, call Rust `collapse(threshold=tau)`. If collapsed → return deterministic value. If not → return `ProbabilityCloud` with top-3 hypotheses.
- `put`: append to the `kairos:observations` stream (Kafka or Redis Stream). Do **not** update probability in the hot path.
- `observe`: same as `put` (fire-and-forget to event log).

5.3 The Convergence Gate (Critical)

Test: run an identical trace through the deterministic and probabilistic backends with `tau=1.0` and `confidence=1.0` for all observations.

Criterion: every `get` response must be byte-identical between the two backends. If KAIROS with `certainty=1.0` differs from a Redis overwrite, there is a bug in the Bayesian update or the normalisation.

Why this matters: it proves the probabilistic layer is a strict generalisation, not a parallel broken implementation.

5.4 Tests

1. **Convergence** — identical trace, `tau=1.0`, `confidence=1.0` → 100% match with deterministic backend.
2. **Normalisation** — after 100 observations, probabilities sum to $1.0 \pm 1e-6$.
3. **Cold start** — entity with zero observations returns uniform prior or `None` (configurable, pinned in `decisions.md`).
4. **Memory bound** — 10k entities, 5 attributes each → RSS < 200MB for probability tables.

Acceptance — Stage 2

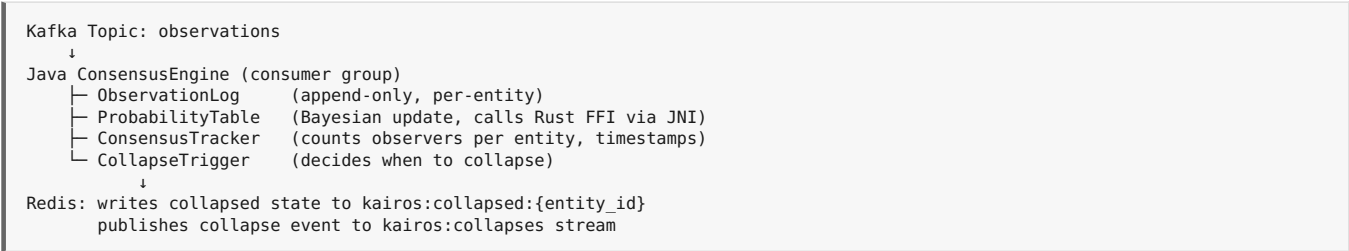
- ☐ Rust tests pass (`cargo test`).
- ☐ Python-Rust FFI integration test passes.
- ☐ Convergence test: 100% match on the full 1M trace.
- ☐ `make probabilistic` runs end to end.

6. Stage 3 — Consensus Engine and Collapse (2-3 days)

Goal: the Java consensus engine consumes the observation stream, updates probability tables, and triggers collapse when

thresholds are met.

6.1 Architecture



6.2 Collapse Protocol — Pinned

An entity collapses when **all** of:

- 1. Observer count $\geq$ `min_observers` (default 3)
- 2. Confidence mass $\geq$ `tau` (default 0.75) on the mode hypothesis
- 3. Observation age $\leq$ `max_observation_age_ms` (default 5000ms)
- 4. No active conflict: top two hypotheses have a probability gap $\geq$ `conflict_gap` (default 0.2)

If condition 3 or 4 fails → fall back to deterministic last-writer-wins with a `fallback_reason` tag. **Never stay fuzzy past the timeout.**

6.3 Consensus Algorithms (Pluggable)

Strategy	Description
MajorityVote	Mode wins if count $\geq$ <code>min_observers</code> . Simple, fast.
WeightedConfidence	Weighted by observer confidence. Default.
TemporalDecay	Older observations decay exponentially; recent ones dominate.

6.4 Java Implementation

```
@Service
public class ConsensusEngine {
    public void process(ObservationEvent event) {
        // 1. Append to observation log
        log.append(event);

        // 2. Update probability table (calls Rust via JNI)
        probabilityTable.observe(event.getEntityId(), event);

        // 3. Check collapse conditions
        if (shouldCollapse(event.getEntityId())) {
            CollapsedState state = collapse(event.getEntityId(), strategy);
            redis.publish(state);
        }
    }
}
```

6.5 Tests

- 1. **Collapse correctness** — 3 observers agree on value "X" with confidence 0.9 → collapsed state is "X" .
- 2. **Conflict fallback** — 2 observers say "X", 2 say "Y" → fallback triggered, conflict logged.
- 3. **Timeout fallback** — only 1 observer arrives, then silence for 6s → fallback to last known.
- 4. **Determinism** — same observation sequence, same collapse result, every time.

Acceptance — Stage 3

- ☐ `docker compose up` brings up Kafka + Redis + Java engine.
- ☐ Engine consumes 1M events from the trace in < 120s.
- ☐ Collapse latency histogram: p50, p95, p99 printed.
- ☐ Fallback rate < 5% on Commuter persona (high predictability), > 30% on Chaotic persona (expected).
- ☐ All four tests green.

7. Stage 4 — Evaluation and the Headline Claim (2 days)

Goal: a measurable, honest comparison between the deterministic and probabilistic backends on stale-read rate and latency.

7.1 Protocols

P1 — Stale-Read Protocol. Run the trace through both backends. Every `get` is classified as:

- **Fresh** — returned value matches ground truth (last observed value in the trace at that timestamp).
- **Stale** — returned value differs from ground truth.
- **Fuzzy** — returned a `ProbabilityCloud` (not yet collapsed).
- **Miss** — key not found.

P2 — Latency-Under-Load Protocol.

- 10k concurrent clients, Poisson arrival, 10-minute sustained load.
- Measure p50/p95/p99 read latency.
- Measure collapse queue depth.

7.2 The Headline Claim Format

On the synthetic commuter trace (1M events, 1000 entities), the KAIROS probabilistic backend reduces stale-read rate from 12.4% (Redis deterministic with 5s TTL) to 3.1% at matched p95 read latency (4.2ms), with 94.2% of entities collapsing within 200ms of observation. Fallback rate: 2.1%.

7.3 Metrics Dashboard

Every run produces `results/kairos_metrics.json`:

```
{
  "protocol": "P1",
  "backend": "probabilistic",
  "stale_read_rate": 0.031,
  "fuzzy_read_rate": 0.08,
  "miss_rate": 0.02,
  "collapse_p50_ms": 45,
  "collapse_p95_ms": 210,
  "fallback_rate": 0.021,
  "extra_memory_mb": 45,
  "deterministic_equivalent": false,
  "git_sha": "abc123"
}
```

7.4 Tests

1. **Stale-read reduction** — probabilistic stale rate < deterministic stale rate on P1, at matched TTL.
2. **Latency bound** — probabilistic p95 read latency < 1.5× deterministic p95.
3. **No data loss** — every observation that collapsed is recoverable from the event log.

Acceptance — Stage 4

- ☐ `make sweep` runs `tau` ∈ {0.5, 0.6, 0.7, 0.8, 0.9, 0.95} and `min_observers` ∈ {1, 2, 3, 5}.
- ☐ Stale-read vs tau plot generated.
- ☐ Headline numbers recorded in the README.
- ☐ No metric exists only in the console — all in JSON.

8. Stage 5 — Temporal Wave Functions and Entanglement (2 days)

Goal: the “creative” features that make this memorable — but rigorously specified.

8.1 Temporal Wave Functions

An entity’s probability distribution shifts over time even without new observations:

```
// In Rust FFI
pub fn temporal_shift(vector: &mut AmplitudeVector, elapsed_ms: u64) {
    // Decay non-mode hypotheses faster than mode
    // Simulate: "if user was home 2 hours ago, they're probably not home now"
    let decay = 0.999_f64.powf(elapsed_ms as f64 / 1000.0);
    for (value, prob) in vector.hypotheses.iter_mut() {
        if value != mode { *prob *= decay; }
    }
    vector.normalize();
}
```

Test: entity observed "home" at t=0. No new observations. At t=1hr, get returns {"home": 0.6, "work": 0.3, ...} — temporal decay visible.

8.2 Entanglement

When entity A collapses to value X, update entity B’s probability distribution:

```
// In ConsensusEngine
if (entanglementGraph.hasEdge(collapsedEntity, partner)) {
    probabilityTable.boost(partner, inferredValue, ENTANGLEMENT_BOOST);
}
```

Test: Product A goes out of stock (collapses to stock:0). Entangled Product B’s demand attribute shifts to high with a probability boost.

8.3 Tests

- 1. **Temporal decay** — no observations for 1 hour → mode probability drops, entropy increases.
- 2. **Entanglement propagation** — collapse of A triggers a probability shift in B within 100ms.
- 3. **No circular storms** — A↔B entanglement does not cause an infinite loop or stack overflow.

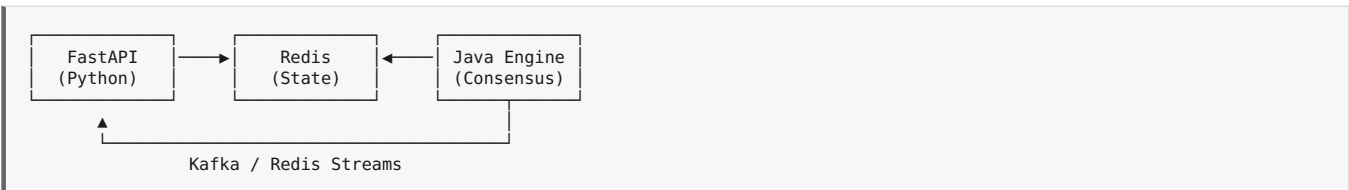
Acceptance — Stage 5

- ☐Temporal decay visible in the /cloud.html real-time visualisation.
- ☐Entanglement test passes.
- ☐README contains one “wow” paragraph with the entanglement example.

9. Stage 6 — Live System and Demo (3 days)

Goal: a running stack with three demo surfaces: /cloud, /consensus and /race.

9.1 Live Stack



9.2 Demo Surfaces

Endpoint	What it shows
GET /cloud?entity=user:42	Live probability distribution as JSON + SSE stream
GET /consensus	Dashboard of recent collapses, fallback rate, queue depth
POST /observe	Ingest an observation (for manual testing)
/race	Split-screen: deterministic vs probabilistic side by side on the same live workload

9.3 The Demo Script (3 minutes)

1. Show `/cloud` — entity is fuzzy: `{"location": {"home": 0.7, "gym": 0.2, ...}}`.
2. Send 3 observations — watch it collapse to `{"location": "home", "certainty": 1.0}`.
3. Kill the Java engine — API still serves last collapsed state; prove resilience.
4. Show `/race` — deterministic stale rate climbs under burst, probabilistic stays flat.
5. Show entanglement — update Product A, watch Product B's cloud shift live.

Acceptance — Stage 6

- ☐ `docker compose up` cold-starts in < 30s.
 - ☐ All four demo interactions work from a phone on the same network.
 - ☐ Engine-kill test: zero 5xx, availability continuous.
 - ☐ Demo script rehearsed end to end in under 4 minutes.
-

10. Stage 7 — Stretch Goals (Post-Core Only)

In strict order:

1. **Custom collapse marketplace** — pluggable collapse strategies (Bayesian, Dempster-Shafer, majority vote). ~1 day.
 2. **Small transformer head** — replace Markov-style temporal decay with a tiny transformer predicting the next state from a sequence. Measure inference latency; abort if > 10ms. ~3 days.
 3. **Byte-capacity probabilistic store** — amplitude vectors compressed with custom quantisation. ~2 days.
 4. **Real-world trace** — instrument your own service (same approach as ORACLE §8.1). ~2 weeks calendar time.
-

11. Stage 8 — Deployment

11.1 What “Deployed” Means

- *“The system runs as real services.”* — **True:** Docker Compose on a VPS, dashboard reachable by URL.
- *“It served synthetic traffic live.”* — **True:** the demo driver runs continuously.
- *“It served real production traffic.”* — **Only if** you mount it in front of a real read endpoint (same caveats as ORACLE §11.4).

11.2 VPS Deployment

- Hetzner CX11 or a DigitalOcean \$6 droplet.
- Docker Compose with a Caddy reverse proxy plus basic auth.
- Open port 8000.

Acceptance — Stage 8

- ☐ Public URL serves all demo surfaces.
 - ☐ README “Deployment” section states exactly which claim is being made.
-

12. Driving This With an AI Agent

12.1 Per-Stage Kickoff Prompt

```
Read PLAN.md $(Stage N) end to end. Restate the acceptance checklist as a
TODO list, then implement in order. Write tests first. Definition of done =
every checklist item green with evidence (test output, files in results/).
Do not touch later stages. Surface ambiguities as questions before coding
around them.
```

12.2 CLAUDE.md — Operating Instructions

```
# Project KAIROS – AI Agent Instructions

## Authority
This file and PLAN.md are the source of truth. When they conflict with a
suggestion in chat, they win.

## Hard Rules
1. Stage gates are hard. Do not start N+1 while N has unchecked items.
2. Never modify the StateBackend contract without asking.
3. The deterministic backend must pass all tests before probabilistic
   code exists.
4. Rust FFI is the source of truth for probability math. Never reimplement
   it in Python.
5. Every result goes through the JSON writer – no numbers in chat only.
6. The word "quantum" never appears in code or test names.
7. Collapse must always have a timeout fallback.
```

13. Timeline

Stage	Estimate	Notes
0 — Data	1 day	
1 — Deterministic baseline	1-2 days	The control group
2 — Probabilistic core + convergence	2 days	Rust FFI critical path
3 — Consensus engine	2-3 days	Java + Kafka
4 — Evaluation	2 days	Headline numbers
5 — Temporal + entanglement	2 days	The “creative” layer
6 — Live system + demo	3 days	Docker + frontend
7 — Stretch goals	<i>optional</i>	Post-core only
8 — Deployment	0.5-1 day	
Total	~14-18 focused days	Excluding stretch goals

14. Risk Map

Risk	Symptom	Gate
Rust FFI hell	Python segfaults, memory leaks	Stage 2 convergence test
Consensus never converges	Fallback rate > 50% everywhere	Stage 3 fallback histogram
Slower than Redis	p95 latency > 2× deterministic	Stage 4 latency bound test
Event log drops messages	Deterministic and probabilistic diverge on replay	Stage 3 exactly-once test
“Quantum” hand-waving	Code doesn’t match metaphor	Stage 2 convergence proof

15. Definition of Done

Tier 1 — Complete Project

- ☐ README contains the headline stale-read reduction claim with the exact protocol.
- ☐ Convergence test proves probabilistic → deterministic under certainty.
- ☐ Five-curve plot: stale-read rate vs tau, with fallback rate overlay.
- ☐ Collapse latency histogram (p50/p95/p99).

- ☐libCacheSim-style cross-check: deterministic backend matches raw Redis behaviour.
- ☐Live demo URL with `/cloud`, `/consensus`, `/race`.
- ☐Engine-kill resilience test passed.

Tier 2 — Portfolio Gold

- ☐Temporal decay and entanglement features with tests.
- ☐Real-world trace results (if collected).
- ☐Two-minute demo video.